

Day 13 — Neural Networks from Scratch

Pure NumPy · No PyTorch · Real understanding

Target: Google · Meta · OpenAI · Anthropic

Why Build From Scratch?

PyTorch can build a neural network in 10 lines. So why spend a whole day in NumPy? Because every interview at Google, Meta, OpenAI, and Anthropic will ask you to explain what happens inside. 'I used PyTorch' is not an answer. Understanding the math is what separates junior from senior ML engineers.

Today you build each piece as a warm-up exercise first, then assemble them into a complete working neural network. No libraries except NumPy.

File	What it does
day13_warmup.py	6 short exercises — build each piece separately and test it
day13_neural_network.py	Full network assembled from all pieces — trains and evaluates

What Is a Neural Network?

A neural network is a series of matrix multiplications with non-linear functions applied between them. That is the complete definition. Everything else is details.

Think of it like a factory assembly line. Raw materials (your input data) go in one end. Each station (layer) transforms the material. The finished product (prediction) comes out the other end. Training adjusts each station until the factory produces the right output.

Layer	What it does	Math
Input	Receives raw data — no computation	X : shape (samples, features)
Hidden 1	Finds low-level patterns	$Z1 = X @ W1 + b1$, $A1 = \text{relu}(Z1)$
Hidden 2	Finds higher-level patterns	$Z2 = A1 @ W2 + b2$, $A2 = \text{relu}(Z2)$
Output	Final prediction	$Z3 = A2 @ W3 + b3$, $A3 = \text{sigmoid}(Z3)$

Weights and Biases — What the Network Learns

A neural network learns by adjusting two things: weights (W) and biases (b). These are just numbers — matrices of floating point values. Training finds the values that make the network produce correct predictions.

Weight matrix W :

W has shape (input_size, output_size). Each value is how strongly one neuron in the previous layer connects to one neuron in the next layer. Large positive = strong positive connection. Near zero = weak connection.

```
# 4 input features, 8 neurons in hidden layer
# W1 shape: (4, 8) – 32 weights total
W1 = np.random.randn(4, 8) * 0.01
# * 0.01 keeps weights small – large initial weights cause unstable training
```

Bias vector b:

b has shape (1, output_size). One bias per neuron. The bias lets a neuron activate even when all inputs are zero.

```
b1 = np.zeros((1, 8)) # initialise to zero – standard practice
```

The Forward Pass — Making a Prediction

The forward pass sends input data through every layer from left to right to get a prediction. Each layer does two operations: a linear step (matrix multiply) and a non-linear step (activation).

Step	Operation	What it does
Linear	$Z = X @ W + b$	Weighted sum of inputs plus bias
Activation	$A = \text{relu}(Z)$	Apply non-linear function
Output	A passed to next layer	Becomes input to next layer

The @ operator means matrix multiplication in NumPy. $X @ W$ multiplies every input by every weight and sums the results. This is the weighted sum — the core operation of every neural network.

Activation Functions — Why Non-Linearity Matters

Without activation functions, stacking layers does nothing. A linear function of a linear function is still linear — 100 layers would collapse into one. Activation functions add non-linearity so the network can learn complex patterns.

Function	Formula	Output range	Used where
ReLU	$\max(0, x)$	0 to infinity	Hidden layers — standard
Sigmoid	$1 / (1 + e^{-x})$	0 to 1	Output: binary classification
Softmax	$e^x / \sum(e^x)$	0 to 1, sums to 1	Output: multi-class
Tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	-1 to 1	Hidden layers — older

ReLU — the most important activation function:

```
def relu(Z):
    # Replaces every negative value with 0
    # relu([-3, -1, 0, 2, 5]) = [0, 0, 0, 2, 5]
```

```
return np.maximum(0, Z)
```

Why ReLU?

1. Simple and fast to compute

2. Gradient is 1 for positive values – no vanishing gradient problem

3. Works well in practice – default choice for hidden layers

Sigmoid — used at the output for binary classification:

```
def sigmoid(Z):  
    # Squashes any value to between 0 and 1  
    # Interpret as probability: 0.9 means 90% chance of class 1  
    # sigmoid([-2, 0, 2]) = [0.119, 0.5, 0.881]  
    return 1 / (1 + np.exp(-Z))
```

Loss Function — Measuring How Wrong We Are

The loss function measures how wrong predictions are. Training minimises this number. For binary classification we use Binary Cross-Entropy Loss.

The formula:

$$\text{Loss} = -1/n * \sum(y * \log(y_pred) + (1-y) * \log(1-y_pred))$$

Intuition: when the true label is 1 (hired), only $y * \log(y_pred)$ matters. If $y_pred = 0.95$ (confident and correct): $-\log(0.95) = 0.05$ — tiny loss. If $y_pred = 0.05$ (confident and wrong): $-\log(0.05) = 3.0$ — huge loss. The formula punishes confident wrong predictions very heavily.

```
def binary_cross_entropy(y_true, y_pred):  
    # Clip to avoid log(0) which is undefined  
    y_pred = np.clip(y_pred, 1e-8, 1 - 1e-8)  
    return float(-np.mean(  
        y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred)  
    ))
```

Backpropagation — How the Network Learns

Backpropagation calculates how much each weight contributed to the error. It uses the chain rule from calculus to send gradients backwards through the network — from output to input. This is the most important algorithm in deep learning.

The intuition: the network made a wrong prediction. Backprop asks which weights were most responsible. It calculates the gradient of the loss with respect to each weight — how much would the loss change if we nudged this weight slightly? Weights that caused more error get larger gradients.

The chain rule:

If y depends on x through z ($y = f(g(x))$), then $dy/dx = dy/dz * dz/dx$. In a network, the loss depends on the output, which depends on layer 2, which depends on layer 1, which depends on the weights. Backprop applies the chain rule from output backwards to input.

Step	Gradient	Formula
Output layer	dZ3	$A3 - y_true$
W3 gradient	dW3	$A2.T @ dZ3 / n$
b3 gradient	db3	$mean(dZ3, axis=0)$
Hidden 2	dZ2	$(dZ3 @ W3.T) * relu_grad(Z2)$
W2 gradient	dW2	$A1.T @ dZ2 / n$
Hidden 1	dZ1	$(dZ2 @ W2.T) * relu_grad(Z1)$
W1 gradient	dW1	$X.T @ dZ1 / n$

Gradient Descent — Updating the Weights

Once backprop gives us the gradients, we update every weight by a small amount in the direction that reduces the loss.

The update rule:

$$W = W - learning_rate * dW$$

`learning_rate` controls the step size. Too large: overshoot the minimum, training diverges. Too small: training is very slow. Typical values: 0.001 to 0.1. We use 0.01 today.

```
# After backprop gives gradients dW1, db1, dW2, db2, dW3, db3:
learning_rate = 0.01

W1 -= learning_rate * dW1 # subtract because we go downhill on loss
b1 -= learning_rate * db1
W2 -= learning_rate * dW2
b2 -= learning_rate * db2
W3 -= learning_rate * dW3
b3 -= learning_rate * db3
```

Warm-Up Exercises — day13_warmup.py

Build and test each component separately before assembling the full network. All 6 exercises go in one file.
Type the code, run it, verify the output matches.

Exercise 1 — ReLU

Expected: `relu([-3, -1, 0, 2, 5]) = [0. 0. 0. 2. 5.]`

Exercise 2 — Sigmoid

Expected: `sigmoid([-2, 0, 2]) = [0.119, 0.5, 0.881]`

Exercise 3 — Binary cross-entropy loss

Expected: perfect predictions give near-zero loss. Random predictions give ~0.693.

Exercise 4 — One forward pass through a single layer

Expected: output shape (5, 8). All values ≥ 0 (ReLU guarantee).

Exercise 5 — Weight initialisation for full network

Expected: $W1=(4,8)$, $W2=(8,4)$, $W3=(4,1)$. Print shapes and total parameter count.

Exercise 6 — One gradient descent step

Expected: weights change in the right direction after one update.

Warm-Up Code:

```
import numpy as np

# EXERCISE 1: ReLU
def relu(Z: np.ndarray) -> np.ndarray:
    return np.maximum(0, Z)

test = np.array([-3.0, -1.0, 0.0, 2.0, 5.0])
print("Ex 1 - ReLU:")
print(f"  Input:    {test}")
print(f"  Output:    {relu(test)}")
print(f"  Expected: [0. 0. 0. 2. 5.]")
print()

# EXERCISE 2: Sigmoid
def sigmoid(Z: np.ndarray) -> np.ndarray:
    return 1 / (1 + np.exp(-Z))
```

```

test2 = np.array([-2.0, 0.0, 2.0])
print("Ex 2 - Sigmoid:")
print(f"  Input:    {test2}")
print(f"  Output:    {sigmoid(test2).round(3)}")
print(f"  Expected: [0.119 0.5   0.881]")
print()

# EXERCISE 3: Binary cross-entropy
def bce(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    y_pred = np.clip(y_pred, 1e-8, 1 - 1e-8)
    return float(-np.mean(
        y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred)
    ))

y_true    = np.array([1.0, 0.0, 1.0, 0.0])
y_perfect = np.array([0.999, 0.001, 0.999, 0.001])
y_random  = np.array([0.5, 0.5, 0.5, 0.5])
print("Ex 3 - Binary Cross-Entropy:")
print(f"  Perfect loss: {bce(y_true, y_perfect):.4f} (near 0)")
print(f"  Random loss:  {bce(y_true, y_random):.4f} (near 0.693)")
print()

# EXERCISE 4: One forward pass
np.random.seed(42)
X_s = np.random.randn(5, 4)
W_s = np.random.randn(4, 8) * 0.01
b_s = np.zeros((1, 8))
print("Ex 4 - Forward Pass:")
Z_s = X_s @ W_s + b_s
A_s = relu(Z_s)
print(f"  Output shape: {A_s.shape} (should be (5, 8))")
print(f"  All >= 0:      {(A_s >= 0).all()}")
print()

# EXERCISE 5: Weight initialisation
np.random.seed(42)
W1 = np.random.randn(4, 8) * 0.01
b1 = np.zeros((1, 8))
W2 = np.random.randn(8, 4) * 0.01
b2 = np.zeros((1, 4))
W3 = np.random.randn(4, 1) * 0.01
b3 = np.zeros((1, 1))
print("Ex 5 - Weight Initialisation:")
print(f"  W1: {W1.shape} W2: {W2.shape} W3: {W3.shape}")
total = W1.size+b1.size+W2.size+b2.size+W3.size+b3.size
print(f"  Total params: {total}")
print()

# EXERCISE 6: Gradient descent step
W_b = np.array([[0.5, -0.3], [0.2, 0.8]])
dW  = np.array([[0.1, 0.2], [0.3, 0.1]])
lr  = 0.1
W_a = W_b - lr * dW
print("Ex 6 - Gradient Descent:")
print(f"  Before: {W_b}")

```

```
print(f" After: {w_a}")
```

Full Neural Network — day13_neural_network.py

Architecture: 4 inputs -> 8 hidden -> 4 hidden -> 1 output. Task: predict hired/not hired. Same dataset as Days 10-12.

```
import numpy as np

# ACTIVATION FUNCTIONS
def relu(Z: np.ndarray) -> np.ndarray:
    return np.maximum(0, Z)

def relu_grad(Z: np.ndarray) -> np.ndarray:
    # Derivative of ReLU: 1 where Z > 0, else 0
    # Used during backpropagation
    return (Z > 0).astype(float)

def sigmoid(Z: np.ndarray) -> np.ndarray:
    return 1 / (1 + np.exp(-Z))

def bce(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    y_pred = np.clip(y_pred, 1e-8, 1 - 1e-8)
    return float(-np.mean(
        y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred)
    ))

# CREATE DATA
np.random.seed(42)
n = 500
age = np.random.randint(22, 60, n).astype(float)
income = np.random.randint(20000, 100000, n).astype(float)
education = np.random.randint(8, 22, n).astype(float)
experience = np.random.randint(0, 20, n).astype(float)
hired = (0.3 * education + 0.5 * experience
         + np.random.normal(0, 2, n) > 10).astype(float)

X = np.column_stack([age, income, education, experience])
y = hired.reshape(-1, 1)

# NORMALISE – neural networks train better when inputs are on similar scales
# (X - mean) / std transforms each feature to mean=0, std=1
X = (X - X.mean(axis=0)) / X.std(axis=0)

# SPLIT
split = int(0.8 * n)
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# INITIALISE WEIGHTS
np.random.seed(42)
W1 = np.random.randn(4, 8) * 0.01
b1 = np.zeros((1, 8))
W2 = np.random.randn(8, 4) * 0.01
b2 = np.zeros((1, 4))
W3 = np.random.randn(4, 1) * 0.01
```



```

b3 = np.zeros((1, 1))

learning_rate = 0.01
epochs = 1000

# TRAINING LOOP
print("Training...")
for epoch in range(epochs):

    # FORWARD PASS
    Z1 = X_train @ W1 + b1    # (400, 8)
    A1 = relu(Z1)             # (400, 8)
    Z2 = A1 @ W2 + b2        # (400, 4)
    A2 = relu(Z2)             # (400, 4)
    Z3 = A2 @ W3 + b3        # (400, 1)
    A3 = sigmoid(Z3)         # (400, 1) predictions

    loss = bce(y_train, A3)

    # BACKWARD PASS
    n_train = X_train.shape[0]

    dZ3 = A3 - y_train
    dW3 = A2.T @ dZ3 / n_train
    db3 = np.mean(dZ3, axis=0, keepdims=True)

    dA2 = dZ3 @ W3.T
    dZ2 = dA2 * relu_grad(Z2)
    dW2 = A1.T @ dZ2 / n_train
    db2 = np.mean(dZ2, axis=0, keepdims=True)

    dA1 = dZ2 @ W2.T
    dZ1 = dA1 * relu_grad(Z1)
    dW1 = X_train.T @ dZ1 / n_train
    db1 = np.mean(dZ1, axis=0, keepdims=True)

    # UPDATE WEIGHTS
    W3 -= learning_rate * dW3
    b3 -= learning_rate * db3
    W2 -= learning_rate * dW2
    b2 -= learning_rate * db2
    W1 -= learning_rate * dW1
    b1 -= learning_rate * db1

    if (epoch + 1) % 100 == 0:
        print(f" Epoch {epoch+1:4d} | Loss: {loss:.4f}")

# EVALUATE
Z1t = X_test @ W1 + b1
A1t = relu(Z1t)
Z2t = A1t @ W2 + b2
A2t = relu(Z2t)
Z3t = A2t @ W3 + b3
A3t = sigmoid(Z3t)

```

```

y_pred = (A3t > 0.5).astype(int)
accuracy = float(np.mean(y_pred == y_test))
print(f"\nTest accuracy: {accuracy:.3f}")
print(f"Correct: {int(accuracy * len(y_test))} / {len(y_test)}")

```

Expected Results

Epoch	Expected loss	What it means
100	~0.60-0.65	Starting to learn
300	~0.50-0.55	Learning main patterns
500	~0.43-0.50	Converging
1000	~0.35-0.45	Reasonably trained
Test accuracy	~78-85%	Good for from-scratch network

If loss does not decrease: check your backprop gradients. If loss goes to NaN: learning rate is too high or weights exploded.

Anki Cards — Day 13

Q: What is a neural network mathematically?

A: A series of matrix multiplications with non-linear activation functions between them

Q: What is a weight matrix W ?

A: Shape (input_size, output_size) — one learnable number per connection between neurons

Q: What does ReLU do?

A: Replaces every negative value with 0 — $\max(0, x)$

Q: Why do we need activation functions?

A: They add non-linearity — without them all layers collapse into one linear function

Q: What is the forward pass?

A: Passing input through every layer in order to produce a prediction

Q: What is backpropagation?

A: Using the chain rule to calculate how much each weight contributed to the error

Q: What is the gradient descent update rule?

A: $W = W - \text{learning_rate} * dW$

Q: What does `relu_grad` return?

A: 1 where $Z > 0$, else 0 — the derivative of ReLU used in backprop

Q: Why normalise inputs before training?

A: Features on different scales cause some weights to dominate — normalising puts them on equal footing

Q: What does `np.clip(y_pred, 1e-8, 1-1e-8)` do?

A: Prevents `log(0)` which is undefined — keeps values just inside $(0, 1)$

Next: Day 14 — PyTorch. You rebuild today's network in PyTorch and see what it automates.